

TRABAJO FINAL DE GRADO

CONCEPTO DE RÚBRICA

Implementación de IA Agéntica para la Creación de Rúbricas

Un caso de estudio aplicado a la materia Organización Empresarial

INTEGRANTES

Renzo Sosa · Emmanuel Avellaneda

Organización Empresarial

Profesor: Alberto Cortez

2026

Resumen

Este trabajo documenta la implementación de una **IA agéntica** (Claude Code) asistida por una **skill** de dominio específico (`rubrica-builder`) para la creación, curado, auditoría y testeo de **rúbricas de evaluación autosuficientes** destinadas al sistema de corrección automática Active-IA. Se parte de un principio estructural —la rúbrica debe ser autosuficiente porque el corrector no vuelve a leer la consigna, y en consecuencia el puntaje debe vivir en el criterio y no en el subcriterio— y se deriva de él un flujo de trabajo completo y reproducible: normalizar la consigna (CURAR), traducirla a criterios (CREAR), validar su estructura, simular correcciones con entregas sintéticas (TEST) y ajustar de forma iterativa (AUDITAR). El flujo se aplica como caso de estudio a las siete rúbricas de la materia Organización Empresarial (TP1–TP6 y TPI). Los resultados de testeo, trazables a archivos concretos del repositorio, muestran que cada criterio evaluado discrimina en la dirección esperada frente a una entrega perfecta. El hallazgo central es que la calidad de una corrección automática depende principalmente de la calidad de la rúbrica —no del modelo que corrige—, lo que justifica dedicar un modo completo del proceso a normalizar la consigna antes de redactar un solo criterio.

Palabras clave: rúbrica de evaluación · corrección automática · IA agéntica · Claude Code · skill de dominio · evaluación educativa · rúbrica analítica.

Abstract

This work documents the implementation of an **agentic AI** (Claude Code) assisted by a domain-specific **skill** (`rubrica-builder`) to create, curate, audit and test **self-contained assessment rubrics** for the automated grading system Active-IA. It builds on a structural principle —a rubric must be self-contained because the grader does not re-read the assignment, so scoring weight must live in the criterion rather than in the sub-criterion— and derives from it a complete, reproducible workflow: normalising the assignment (CURATE), translating it into criteria (CREATE), validating its structure, simulating corrections with synthetic submissions (TEST) and refining iteratively (AUDIT). The workflow is applied as a case study to the seven rubrics of the course *Organización Empresarial* (TP1–TP6 and the final integrative project). The test results, traceable to concrete files in the repository, show that every evaluated criterion discriminates in the expected direction against a perfect

submission. The central finding is that the quality of an automated correction depends primarily on the quality of the rubric —not on the model doing the grading— which justifies devoting a whole stage of the process to normalising the assignment before writing a single criterion.

Keywords: assessment rubric · automated grading · agentic AI · Claude Code · domain skill · educational assessment · analytic rubric.

Índice

Resumen

1. Introducción

Objetivos específicos

2. Marco teórico

¿Qué es una rúbrica de evaluación?

¿Qué es una IA agéntica?

¿Qué es una skill?

3. Instalación y configuración del entorno

Instalación de la IA agéntica (Claude Code)

Instalación de la skill `rubrica-builder`

4. Normalización de la consigna y de la entrega

Caso de estudio: estandarizar la tecnología antes de armar la rúbrica

5. Creación de la rúbrica (modo CREAR)

6. Auditoría de la rúbrica (modo AUDITAR)

7. Validación estructural de la rúbrica

8. Testeo de la rúbrica (modo TEST)

9. Ajuste iterativo de consigna, rúbrica y entrega

Ejemplo de iteración real: rúbrica del TP3 (v1 → v2)

10. Caso de estudio aplicado: Organización Empresarial

11. Conclusiones finales

12. Fuentes utilizadas y reconocimiento de autoría

1. Introducción

El presente trabajo tiene como objetivo documentar la implementación de una **IA agéntica** —en este caso, **Claude Code**— junto con una extensión de conocimiento específica denominada **skill**, para la creación, curado, auditoría y testeo de **rúbricas de evaluación autosuficientes**, en el marco del sistema de corrección automática **Active-IA**.

A diferencia de un uso conversacional simple de un modelo de lenguaje (preguntar y recibir una respuesta), una IA agéntica es capaz de **ejecutar un proceso de varios pasos, tomar decisiones intermedias, usar herramientas** (scripts, archivos, terminal) y **validar sus propios resultados** antes de entregarlos. Este trabajo toma como skill de referencia `rubrica-builder`, desarrollada por Juan Cruz Robledo, y la utiliza como caso de aplicación práctica para transformar la consigna de un Trabajo Práctico en una rúbrica de evaluación estructurada, coherente y —sobre todo— **corregible por otra IA sin ambigüedad**.

Como caso de estudio transversal a lo largo de todo el informe se utilizan las rúbricas reales elaboradas para la materia **Organización Empresarial (OE)**, ya que es el contexto en el que se origina este trabajo.

Objetivos específicos

1. Describir el funcionamiento de una IA agéntica y el rol de una skill de dominio específico para extender sus capacidades a un área concreta.
2. Documentar el proceso completo de creación de una rúbrica autosuficiente para corrección automática, desde la normalización de la consigna hasta el testeo con entregas sintéticas.
3. Identificar los límites técnicos del corrector automático Active-IA y establecer criterios de normalización que garanticen la corregibilidad de los trabajos prácticos.
4. Validar empíricamente que las rúbricas producidas discriminan correctamente entre entregas con distintos niveles de cumplimiento, mediante correcciones simuladas con entregas sintéticas.
5. Registrar el proceso aplicado en la materia Organización Empresarial como caso de estudio documentado y reproducible, trazable a archivos concretos.

Alcance y tipo de trabajo

Conviene delimitar desde el inicio la naturaleza de este trabajo para leerlo con el criterio adecuado. No se trata de una **tesis de investigación empírica** con hipótesis, muestreo probabilístico y análisis estadístico inferencial, sino de un **informe técnico de implementación**: documenta, con rigor y de forma reproducible, cómo se aplicó una herramienta concreta a un problema real de la cursada. En consecuencia, la palabra "validar" se emplea a lo largo del texto en el sentido de *comprobar el comportamiento esperado mediante casos de prueba diseñados a propósito* (a la manera de un test de software), y no en el sentido estadístico de validación de un constructo sobre una muestra. Los resultados de las simulaciones de corrección (§8) tienen, por su método, carácter **indicativo** y no probatorio; se declara explícitamente su alcance en cada tabla.

Video de la tesis (fundamentos y motivaciones):
<https://www.youtube.com/watch?v=OOzTbtUt-Q>. *Nota: el presente informe es autocontenido; el video amplía motivaciones pero no es requisito para comprenderlo.*

2. Marco teórico

2.1. ¿Qué es una rúbrica de evaluación?

Una rúbrica es un instrumento de evaluación que descompone un trabajo práctico en **criterios** (grandes bloques evaluables, cada uno con un peso o ponderación) y **subcriterios** (verificaciones puntuales dentro de un criterio, sin nota propia). Cada subcriterio se apoya en **evidencias**: afirmaciones verificables de forma binaria (cumple / no cumple) mirando únicamente la entrega del alumno.

Esta estructura se corresponde con lo que la literatura de evaluación educativa denomina **rúbrica analítica** —que descompone el desempeño en dimensiones evaluadas por separado—, por oposición a la **rúbrica holística**, que asigna una única valoración global (Brookhart, 2013). El enfoque analítico es el que adopta este trabajo, por ser el único compatible con una corrección automática por criterios independientes. Diversos autores coinciden, además, en que explicitar los criterios de evaluación por anticipado no solo ordena y agiliza la calificación, sino que orienta el propio aprendizaje del estudiante (Andrade, 2000; Stevens & Levi, 2013).

Contenido ampliado en el video "¿Qué es una rúbrica?", disponible en: drive.google.com/drive/folders/1WewN_uwqNbvnYhkBqQ4WZ2_lge4UHPCU (referenciado también en la sección de fuentes).

El punto central que este trabajo busca remarcar —y que constituye el eje conceptual de toda la skill— es que una rúbrica pensada para ser corregida por una IA debe ser **autosuficiente**: en el momento de la corrección automática, el modelo que corrige **no vuelve a leer la consigna original**. Solo tiene ante sí dos elementos: la rúbrica y la entrega. En consecuencia, todo requisito de la consigna que no haya quedado plasmado explícitamente en la rúbrica **directamente no se evalúa**, sin importar cuán importante fuera para el docente.

De esta premisa se deriva la segunda regla estructural que gobierna el diseño de una rúbrica: **el puntaje vive en el criterio, no en el subcriterio**. Si un requisito necesita poder penalizarse de forma independiente, debe modelarse como **criterio propio** con su peso, y no quedar diluido dentro de un criterio más amplio. De lo contrario, el corrector detecta la falla pero no tiene "de dónde restar" sin perjudicar a otros requisitos agrupados en el mismo criterio, lo cual vuelve la corrección **no**

determinista (la misma entrega podría obtener notas distintas según el modelo o la corrida).

2.2. ¿Qué es una IA agéntica?

Una IA agéntica es un sistema basado en un modelo de lenguaje que, a diferencia de un chatbot tradicional, tiene la capacidad de:

- Interpretar un objetivo complejo y **planificar los pasos** necesarios para alcanzarlo.
- **Usar herramientas** del entorno en el que corre (leer y escribir archivos, ejecutar scripts, correr comandos de terminal, consultar documentación).
- **Verificar su propio trabajo** antes de darlo por terminado (por ejemplo, corriendo un script de validación y corrigiendo el resultado si falla).
- Operar en **múltiples turnos**, sosteniendo contexto y tomando decisiones intermedias sin que el usuario deba especificar cada micro-paso.

Este patrón —intercalar pasos de razonamiento con acciones que invocan herramientas externas— fue formalizado en la literatura bajo el paradigma **ReAct** (Yao et al., 2023), y la arquitectura general de los agentes autónomos basados en modelos de lenguaje se encuentra sistematizada en el relevamiento de Wang et al. (2024). En el ámbito educativo, tanto las oportunidades como los riesgos de incorporar modelos de lenguaje a tareas docentes han sido documentados de forma crítica (Kasneci et al., 2023; Zawacki-Richter et al., 2019), marco general en el que se inscribe este trabajo.

En este trabajo la IA agéntica utilizada es **Claude Code**, de Anthropic (Anthropic, 2024), un agente de línea de comandos pensado originalmente para tareas de programación, pero cuya arquitectura de "modelo + herramientas + instrucciones" es perfectamente aplicable a un dominio no estrictamente técnico como la elaboración de instrumentos de evaluación.

El agente, por sí solo, no "sabe" cómo se arma una rúbrica válida para el sistema Active-IA. Ese conocimiento específico del dominio se le agrega a través de una **skill**, concepto que se desarrolla a continuación.

2.3. ¿Qué es una skill?

Una skill es un **paquete de conocimiento especializado** que se instala sobre la IA agéntica para extender sus capacidades a un dominio concreto. En términos generales es un **patrón de diseño** —conocimiento de dominio empaquetado para que un agente lo cargue bajo demanda cuando el contexto lo requiere—, independiente del producto concreto que lo implemente; en este trabajo se materializa mediante el mecanismo de skills de Claude Code. Técnicamente se compone de:

- Un archivo `SKILL.md`, con una descripción (usada por el agente para saber *cuándo* activarse) y las instrucciones detalladas de *cómo* ejecutar la tarea.
- Archivos de referencia (`references/`) con el conocimiento de dominio que el agente debe respetar como fuente de verdad (por ejemplo, el esquema exacto que debe tener el JSON de una rúbrica, o los límites técnicos de lo que el corrector automático puede o no puede leer).
- Scripts auxiliares (`scripts/`) que el propio agente ejecuta para **validar** o **simular** su trabajo, en lugar de confiar únicamente en su propio criterio.
- Ejemplos (`assets/` , `examples/`) que sirven de molde.

La skill utilizada en este trabajo, `rubrica-builder`, define **cuatro modos de funcionamiento**:

- **CURAR** (§4): analiza la consigna original *antes* de escribir criterios, detecta requisitos incompatibles con el corrector automático y propone adaptaciones.
- **CREAR** (§5): traduce la consigna ya curada en una rúbrica estructurada con criterios, subcriterios y evidencias binarias.
- **AUDITAR** (§6): revisa una rúbrica existente contra la consigna original, detectando contradicciones, omisiones, diluciones e invenciones.
- **TEST** (§8): simula correcciones reales con entregas sintéticas para verificar que cada criterio discrimina como se espera antes de subir la rúbrica al sistema real.

3. Instalación y configuración del entorno

3.1. Instalación de la IA agéntica (Claude Code)

Claude Code se instala como una herramienta de línea de comandos (CLI). Los pasos generales son:

1. Contar con **Node.js** instalado (requisito de la herramienta de instalación).
2. Instalar el CLI mediante el gestor de paquetes `npm`.
3. Autenticarse con una cuenta de Anthropic (API key o suscripción, según el plan).
4. Verificar la instalación ejecutando el comando del agente dentro de una carpeta de trabajo (en este caso, la carpeta del repositorio de la materia Organización Empresarial).

3.2. Instalación de la skill `rubrica-builder`

La skill se instala dentro del entorno del agente mediante el comando indicado en su propia documentación:

```
npx skills add https://github.com/JuanCruzRobledo/rubrica-builder
```

Este comando descarga la skill y la deja disponible para que el agente la cargue **automáticamente** cuando detecta, por el pedido del usuario, que la tarea corresponde a alguno de sus cuatro modos (por ejemplo: *"armá los criterios para este TP"*, *"revisá si esta rúbrica cubre todo lo que pide el parcial"*, *"probá cómo corrige esta rúbrica con esta entrega"*). No es necesario invocarla por nombre explícitamente: el agente la reconoce por el contexto del pedido.

Requisitos técnicos declarados por la skill:

- Python 3.10 o superior (usa únicamente librería estándar, sin dependencias externas) — necesario para los scripts de validación y testeo.
- El CLI de Claude Code disponible en el `PATH` del sistema — necesario específicamente para el modo TEST, que dispara una corrección real con el modelo.

Para **verificar** que la skill quedó correctamente instalada, basta con pedirle al agente que liste las skills disponibles, o bien iniciar cualquier pedido que active alguno de sus

cuatro modos (por ejemplo, *"armá los criterios para este TP"*): si la instalación fue exitosa, `rubrica-builder` aparece entre las skills reconocidas y el agente responde ejecutando su flujo —lectura de la consigna, propuesta de criterios, validación— en lugar de una respuesta conversacional genérica. Este chequeo cumple el paso 4 de §3.1 sin depender de la salida textual exacta del comando de instalación, que puede variar según la versión del gestor de skills.

4. Normalización de la consigna y de la entrega

Este es, en la práctica, el paso más subestimado y el que más impacto tiene sobre la calidad final de la rúbrica. Antes de generar cualquier criterio, hay que preguntarse: **¿esta consigna, tal como está redactada, se puede corregir con una IA?**

El motivo es que el corrector automático de Active-IA tiene **límites duros** que no dependen de qué tan bien esté escrita la rúbrica:

- 1. **No navega a recursos externos.** Un link a un repositorio de GitHub, una URL de un deploy o un video en YouTube/Drive nunca se abren: son invisibles para el corrector.
- 2. **Filtra archivos por extensión según el modo de corrección** (`solo_codigo` , `web_completo` , `proyecto_completo` o `personalizado`). Las imágenes (`.png` , `.jpg` , `.jpeg` , `.gif` , etc.) están explícitamente excluidas en los modos de código: **nunca llegan al corrector**.
- 3. **Es un solo flujo: código O informe en PDF, nunca ambos** en la misma corrección.
- 4. **No ejecuta nada.** El corrector lee archivos estáticos; no corre el programa, no verifica que un deploy responda, no corre una batería de tests.

De este diagnóstico surge el **modo CURAR** de la skill, cuya función es analizar la consigna original y, por cada punto que rompa alguno de los cuatro límites anteriores, proponer uno de tres destinos:

Tabla 1. Los tres destinos posibles de un requisito de la consigna en el modo CURAR.

Destino	Cuándo se aplica	Ejemplo
Adaptar (preferido)	El requisito se puede reformular para que entre en el flujo único, sin perder su intención pedagógica	"Adjuntá el link del repo" → "Entregá un .zip del proyecto con el README mostrando la URL del repo"
Mantener + corrección manual	El requisito es importante pero no admite adaptación (ej. un video de defensa oral)	Se deja igual en la consigna, se marca como corrección manual, y no entra en la rúbrica

Sacar (solo de la rúbrica)	El punto es accesorio para la nota (un link decorativo)	No se escribe como criterio; la consigna lo conserva igual
-----------------------------------	---	--

Un principio importante que distingue este modo del resto: **la consigna nunca pierde contenido** por decisión unilateral del agente. Lo que se "saca" es exclusivamente de la rúbrica, mientras que la consigna se adapta o se conserva, y el proceso es siempre conversacional: el agente propone una cura concreta y pregunta antes de aplicarla.

Este es, precisamente, el criterio pedagógico que se aplicó al confeccionar las rúbricas de Organización Empresarial que se documentan en la sección 10: al ser una materia no estrictamente de programación, se decidió de antemano que todas las entregas de OE se resuelven en **texto plano**, evitando de raíz pedir capturas de pantalla, diagramas en imagen o links externos. Cuando la consigna original requería representar un esquema, se optó por describirlo en texto o, en materias con contenido técnico afín, expresarlo en formatos de texto estructurado (por ejemplo, notación Mermaid o XML) en lugar de una imagen.

4.1. Caso de estudio: estandarizar la tecnología antes de armar la rúbrica (evaluación de control de versiones con Git)

El paso de normalización no se agota en el formato del archivo de entrega: también implica decidir, **antes** de escribir un solo criterio, qué tecnología concreta va a usar toda la cursada para resolver una consigna, de manera que exista un único formato de evidencia posible para auditar. El siguiente caso, documentado en video (ver Fuentes, §12), ilustra este razonamiento aplicado a un TP cuyo objetivo es evaluar el uso de control de versiones.

El punto de partida es que el TP ideal para un corrector automático es aquel que se resuelve enteramente con un PDF de preguntas y respuestas: es el formato más fácil de auditar, porque el corrector solo necesita leer texto. Pero no todos los temas de la cursada se prestan a ese formato. Un TP sobre control de versiones, por definición, no puede evaluarse solo con teoría: hay que verificar que el alumno efectivamente creó ramas, hizo commits, aplicó un merge. Ahí aparece la primera decisión de normalización: **unificar la tecnología**. Se optó por Git como sistema de control de versiones para toda la cursada, de modo que los alumnos entregaran evidencia comparable entre sí en lugar de un mosaico de herramientas distintas.

La segunda decisión fue el formato de esa evidencia. La opción más intuitiva —pedir capturas de pantalla de la terminal mostrando cada comando ejecutado— se descartó deliberadamente: como ya se estableció en los cuatro límites duros descritos al inicio de esta sección, las imágenes quedan excluidas en los modos de código y, aunque no lo estuvieran, una IA no es un método confiable para verificar a partir de una captura que un commit, una rama o un merge realmente existen en el historial del repositorio. Una captura se puede editar, recortar o simular; un repositorio Git es sustancialmente más difícil de falsificar — reescribir el historial requiere acceso a la carpeta `.git`, conocimiento de comandos como `git rebase` o `git filter-branch`, y la voluntad de alterar metadatos (fechas, autores, hashes) de forma coherente. En el contexto de una entrega universitaria típica, esto lo convierte en una evidencia de auditoría mucho más robusta que cualquier captura de pantalla.

Consultando la documentación oficial de GitHub sobre la estructura interna de un repositorio, se identificó que la carpeta oculta `.git` —generada automáticamente por `git init`— contiene el registro de commits, las referencias a las ramas (`refs/heads/`) y la configuración de identidad local. Es, en los hechos, el equivalente a un log de auditoría que no depende de lo que el alumno afirme haber hecho, sino de lo que el propio sistema de control de versiones registró.

Conviene, sin embargo, acotar el alcance de esta afirmación: la carpeta `.git` **no es inviolable**. Un alumno con conocimiento suficiente puede alterar la fecha de un commit (`--date` , `GIT_COMMITTER_DATE`), falsear la identidad del autor (`git config user.name`) o reconstruir por completo un historial (`rebase` , `filter-branch`) para simular un proceso de trabajo que nunca ocurrió. Lo que se sostiene aquí es una afirmación más modesta y más defendible: **falsificar un historial de Git coherente es sustancialmente más difícil que trucar una captura de pantalla**, y exige precisamente el dominio de las herramientas de versionado que el TP busca evaluar. La evidencia no es incorruptible, pero su costo de falsificación es lo bastante alto —y su superficie de verificación lo bastante amplia (mensajes, orden, ramas, merges, identidad)— como para volverla el mejor artefacto disponible dentro de los límites del corrector automático.

La consigna original del TP1 (que pedía un PDF de teoría más capturas de pantalla de la Parte 2) se modificó en consecuencia: la entrega pasó a ser un único `.zip` que contiene el PDF con la resolución de la parte teórica y **la carpeta `.git`** del repositorio local, tal como se documenta en la consigna vigente del TP1 (§10) y se verifica en el criterio C1 de su rúbrica. Este caso resume el principio general de esta sección: antes de

escribir un criterio hay que analizar qué es lo que realmente se necesita evaluar, reconocer las limitaciones concretas de la IA correctora, y recién entonces decidir qué tecnología y qué formato de entrega van a usar todos los alumnos por igual.

5. Creación de la rúbrica (modo CREAR)

Con la consigna ya normalizada (curada), el modo CREAR traduce el texto en una rúbrica completa. El proceso que ejecuta el agente es:

1. **Extracción de requisitos.** Lectura completa de la consigna, listando todo lo evaluable: funcionalidades o actividades pedidas, restricciones, condiciones de entrega, estructura obligatoria y prohibiciones. Todo requisito explícito debe terminar reflejado en algún criterio o evidencia; lo que no está en la consigna, no entra en la rúbrica.
2. **Agrupación en criterios según la "palanca de descuento".** No hay un número fijo de criterios: la cantidad la determina la cantidad de aspectos que el docente necesita poder penalizar de forma independiente. El **test de granularidad** es simple: *¿este requisito necesita bajar la nota por sí solo, sin afectar a otros?* → *criterio propio*. *¿Es solo una evidencia de verificación de algo más grande?* → *subcriterio*. A cada criterio se le asigna un peso proporcional a la importancia que la consigna le otorga, y la suma de todos los pesos debe dar exactamente **100**.
3. **Subcriterios con evidencias verificables.** Cada subcriterio se apoya en evidencias redactadas como afirmaciones binarias y objetivas, nunca subjetivas.
4. **Penalizaciones y condiciones de desaprobación,** solo si la consigna las establece explícitamente.
5. **Validación obligatoria** (ver sección 7) antes de dar la rúbrica por terminada.

Ejemplo aplicado (TP2 de Organización Empresarial)

Como ejemplo concreto, se tomó la consigna del **TP2 — Ingeniería de Prompts y Pensamiento Computacional**, que pide al alumno resolver 6 actividades relacionadas con técnicas de diseño de instrucciones para modelos de lenguaje. La rúbrica resultante (`Pruebas Empiricas/TP2/rubrica-tp2-ingenieria-prompts.json`) quedó estructurada en 6 criterios —uno por actividad—, cada uno con su propio peso y sus `instrucciones_puntuacion` explícitas. A modo ilustrativo, el primer criterio quedó modelado así:

Listado 1. Criterio C1 de la rúbrica del TP2, con su peso, sus instrucciones de puntuación y sus subcriterios de evidencias binarias.

```

{
  "id": "C1",
  "nombre": "Actividad 1: Arquitectura de la Instrucción",
  "peso": 18,
  "instrucciones_puntuacion": "Si alguno de los 4 elementos fundamentales
    está ausente, techo de 60% (10.8 pts). Si no hay delimitadores
    visibles entre secciones, descontar 25% (4.5 pts). Si el prompt no
    menciona las 3 categorías de clasificación requeridas, descontar 20%
    (3.6 pts). Los descuentos son acumulables.",
  "subcriterios": [
    { "id": "C1.1",
      "descripcion": "Los 4 elementos fundamentales (Instrucción, Contexto,
        Datos de Entrada, Indicador de Salida) están presentes y claramente
        identificados en el prompt.",
      "evidencias": [
        "El prompt incluye una sección que funciona como Instrucción (indica
          la tarea que debe realizar el modelo)",
        "El prompt incluye una sección que funciona como Contexto (describe
          el escenario, rol del asistente o información de fondo)",
        "El prompt incluye una sección que funciona como Datos de Entrada
          (muestra o define el correo a clasificar)",
        "El prompt incluye una sección que funciona como Indicador de Salida
          (especifica el formato o estructura de la respuesta esperada)"
      ]
    },
    { "id": "C1.2",
      "descripcion": "El prompt usa delimitadores visibles para separar
        las secciones entre sí.",
      "evidencias": [
        "Se identifican caracteres o marcadores delimitadores (como ---,
          ###, ===, <>, comillas triples u otros) entre al menos dos
          secciones del prompt"
      ]
    },
    { "id": "C1.3",
      "descripcion": "El prompt está orientado al caso pedido: clasificar
        correos de soporte técnico en las 3 categorías específicas.",
      "evidencias": [
        "El prompt menciona las 3 categorías: 'Urgente', 'Consulta General'
          y 'Feedback'",
        "El prompt está dirigido a un asistente de IA para clasificar
          correos electrónicos de soporte técnico"
      ]
    }
  ]
}

```

```
]
}
```

Nótese cómo cada subcriterio (C1.1, C1.2, C1.3) es un checklist de evidencias sin nota propia, mientras que la nota completa de la actividad depende del peso (18 puntos) del criterio C1, ajustado hacia abajo mediante reglas numéricas explícitas de descuento — exactamente el principio de granularidad descrito en la sección 2.1.

La distribución completa de pesos de la rúbrica del TP2 es verificable: C1 (18) + C2 (18) + C3 (18) + C4 (16) + C5 (16) + C6 (14) = **100 puntos**. Esta suma es validada automáticamente por `scripts/validar_rubrica.py` antes de subir la rúbrica al sistema (ver §7).

6. Auditoría de la rúbrica (modo AUDITAR)

Este modo no figuraba en el esquema orientativo inicial de este informe, pero se incorpora porque es uno de los **cuatro modos centrales** de la skill y cumple un rol de control de calidad que ninguno de los otros pasos cubre: revisar una rúbrica ya redactada (propia o de un tercero) contra la consigna original, con mirada de auditor.

El agente revisa, en este orden:

1. **Contradicciones** entre lo que pide la consigna y lo que premia o exige la rúbrica.
2. **Omisiones**: requisitos explícitos del TP que la rúbrica no evalúa. Es la falla más grave, porque lo omitido nunca se corrige.
3. **Requisitos diluidos**: puntos importantes que sí están en la rúbrica pero enterrados como subcriterio sin peso propio.
4. **Inversiones**: criterios o concesiones que la rúbrica agrega pero que la consigna no respalda en ningún lado.
5. **Elementos visuales** de la consigna que no se tradujeron a texto dentro de la rúbrica.
6. **Reajuste de pesos** a 100, si se agregaron o quitaron criterios.

El resultado de este modo es un breve resumen de hallazgos (qué se corrigió y por qué) seguido de la rúbrica corregida, lista para volver a validar.

Ejemplo aplicado: hallazgo de auditoría en el TP4

Durante la revisión de la rúbrica del TP4 (Ecosistema de una startup) el modo AUDITAR detectó que un subcriterio sobre la **propuesta de valor** estaba enterrado dentro de un criterio más amplio llamado "Modelo de negocio" (C3, peso 20), sin peso propio. Como la consigna establecía que la propuesta de valor era el eje de evaluación de toda la unidad, cualquier alumno que la omitiera o la expresara mal solo perdía fracción de los 20 pts de C3, junto con otros ítems del modelo de negocio que podían compensar la falla. El hallazgo textual del agente fue:

"C3.1 (Propuesta de valor) está en rúbrica como subcriterio sin peso, pero la consigna dice 'la propuesta de valor es el núcleo del TP'. Requiere criterio

propio. Acción: extraer C3.1 como nuevo criterio C5 con peso 15; redistribuir C3 a 15 pts. Suma verificada: $15+18+20+15+17+15 = 100$. "

Tras aplicar el ajuste y volver a ejecutar `validar_rubrica.py`, la suma de pesos fue correcta y el criterio quedó con capacidad independiente de penalizar la omisión de la propuesta de valor.

7. Validación estructural de la rúbrica

Independientemente del modo utilizado, ninguna rúbrica se entrega sin pasar antes por el script `scripts/validar_rubrica.py`, incluido en la skill:

```
python scripts/validar_rubrica.py <ruta_al_json>
```

Este script replica en local las reglas que aplica el backend real de Active-IA: suma de pesos igual a 100, IDs con el patrón correcto, ausencia de evidencias vacías, nombres de campo correctos (por ejemplo, `nota_maxima` y no `nota_final`, un error presente incluso en cierta documentación desactualizada del propio sistema). Si el script falla, indica exactamente qué corregir; si pasa, imprime la confirmación "  **RÚBRICA VÁLIDA**", que se adjunta como evidencia en el informe final de cada rúbrica entregada. Estos informes de validación, junto con las corridas de testeo, quedan versionados en el repositorio dentro de la carpeta de cada TP en `Pruebas Empiricas/`, de modo que la confirmación es trazable y auditable como anexo del proceso.

Este paso es barato de ejecutar y evita el rechazo de la rúbrica al cargarla en el sistema real.

8. Testeo de la rúbrica (modo TEST)

Que una rúbrica sea **estructuralmente válida** no garantiza que **corrija bien**. El modo TEST cierra esa brecha: simula, en el entorno local, el pipeline real de corrección (consolidación de la entrega + armado del prompt de corrección), disparando la evaluación con el CLI de Claude en lugar del flujo de producción (N8N → Gemini). Los resultados son indicativos, pero permiten detectar problemas de diseño de la rúbrica antes de subirla al sistema real.

```
python scripts/simular_correccion.py \
  --rubrica <rubrica.json> \
  --entrega <archivo|carpeta|.zip|.txt> \
  --materia "Organización Empresarial" \
  --alumno "Alumno de prueba" \
  --modo solo_codigo
```

Para cada rúbrica se generan entregas **sintéticas** diseñadas a propósito para fallar en un único criterio a la vez (`s1_falla_c1` , `s2_falla_c2` , etc.), además de una entrega perfecta (`s0_perfecta`) y, en algunos casos, entregas con cumplimiento parcial. La lógica es la de un test unitario aplicado a una rúbrica: si el criterio C_n está bien diseñado, la entrega que falla exactamente en C_n debe producir una nota visiblemente más baja que la entrega perfecta, y el feedback generado debe señalar puntualmente esa falla.

Criterio de éxito en las tablas de resultados

El símbolo  en la columna "Resultado" indica que el criterio **discriminó en la dirección correcta**: la entrega con la falla simulada obtuvo una nota inferior a la entrega perfecta, y el feedback identificó la falla esperada. No indica que la nota obtenida coincida exactamente con el valor de la columna "Nota esperada", que es una estimación orientativa basada en el peso del criterio afectado. Cuando la nota obtenida se desvía significativamente (por ejemplo, `s4_falla_c4` : obtenida 47 frente a orientativa ~84), eso generalmente refleja que el criterio activa una **condición de desaprobación o techo severo** definido en `instrucciones_puntuacion`, lo cual es comportamiento intencionado. Los resultados de este modo son **indicativos**: se generan con el

CLI de Claude y no con el pipeline real (N8N → Gemini), por lo que la nota final en producción puede diferir.

Resultados obtenidos (TP2 — Organización Empresarial)

Tabla 2. Testeo del TP2: 8 entregas sintéticas (una por criterio más cumplimiento parcial).

Entrega sintética	Qué testea	Dirección esperada	Nota obtenida	Resultado
s0_perfecta	Rúbrica completa	≥ 90	100	✓
s1_falla_c1	C1 discrimina (18 pts)	< 100	91.9	✓
s2_falla_c2	C2 discrimina (18 pts)	< 100	82	✓
s3_falla_c3	C3 discrimina (18 pts)	< 100	89.2	✓
s4_falla_c4	C4 discrimina (16 pts) — activa techo severo	< 100	47.0	✓ (techo intencionado)
s5_falla_c5	C5 discrimina (16 pts)	< 100	87	✓
s6_falla_c6	C6 discrimina (14 pts)	< 100	87.5	✓
s7_parcial_c1	C1 con cumplimiento parcial	< 100 y > falla total	88.3	✓

Fuente: `Pruebas Empíricas/TP2/resumen.json`, generado por el script de testeo de la skill sobre la rúbrica del TP2.

Se observa que, en todos los criterios, la nota obtenida discrimina en la dirección correcta respecto de la entrega perfecta. El caso de `s4_falla_c4` (nota 47.0) es un ejemplo de **criterio troncal**: al fallar, activa un techo de puntuación severo definido en `instrucciones_puntuacion`, comportamiento intencional cuando ese requisito es central. Este tipo de desvíos motiva el ajuste iterativo (§9).

Resultados obtenidos (TP1 — Primeros pasos con Git)

Tabla 3. Testeo del TP1: criterio teórico clave y condición de desaprobación por ausencia de `.git`.

Entrega sintética	Qué testea	Nota esperada	Nota obtenida	Resultado
so_perfecta	Rúbrica completa	100	100	✓
s1_falla_c2	C2 discrimina (10 pts) — pregunta teórica sin responder	90	90	✓
s2_sin_git_cd2	CD2 discrimina — ausencia de la carpeta <code>.git</code>	≤30 (techo)	30	✓

Fuente: *Pruebas Empíricas/TP1/*.

El caso `s2_sin_git_cd2` confirma el diseño intencional descrito en §4.1: la carpeta `.git` es el artefacto que respalda toda la parte práctica (C6-C9). Sin ella, esos cuatro criterios no tienen evidencia verificable (quedan en 0) y, adicionalmente, la condición de desaprobación `CD2` impone un techo de 30 puntos aunque la parte teórica (C2-C5) esté perfecta. El subtotal bruto antes del techo fue 42/100, lo que evidencia una redundancia de diseño: la ausencia de `.git` ya se penaliza dentro de C1 (−8 puntos) y además dispara el techo global de CD2, que termina siendo el factor decisivo.

Resultados obtenidos (TP5 — Live Coding)

Tabla 4. Testeo del TP5: criterio conceptual clave y condición de desaprobación por formato de entrega inválido.

Entrega sintética	Qué testea	Nota esperada	Nota obtenida	Resultado
so_perfecta	Rúbrica completa	100	100	✓
s1_falla_c4	C4 discrimina (18 pts) — define Boilerplate sin mencionar Snippets	91	91	✓
s2_zip_invalido_cd1	CD1 discrimina — entrega en <code>.rar</code> en vez de <code>.zip</code>	0 (techo)	0	✓

Fuente: *Pruebas Empíricas/TP5/*.

El caso `s2_zip_invalido_cd1` es el ejemplo más contundente de una condición de desaprobación: aunque el contenido teórico interno era correcto y completo en las cinco consignas, el sólo hecho de comprimir en `.rar` en vez de `.zip` —algo que la consigna prohíbe explícitamente— anula la nota por completo. Es la penalización más severa observada en todo el testeo, y la más evitable: un error de empaquetado, no de contenido.

Resultados obtenidos (TP6 — Automatización con n8n)

Tabla 5. Testeo del TP6: coherencia entre nodos del workflow y condición de desaprobación por ausencia del `.json` exportado.

Entrega sintética	Qué testea	Nota esperada	Nota obtenida	Resultado
so_perfecta	Rúbrica completa	100	100	✓
s1_falla_c7	C7 discrimina (15 pts) — nombre de campo incorrecto en Edit Fields	~95 (solo C7)	91	✓
s2_sin_json_cd2	CD2 discrimina — ausencia del <code>.json</code> exportado de n8n	≤30 (techo)	30	✓

Fuente: `Pruebas Empíricas/TP6/`.

El caso `s1_falla_c7` arrojó un hallazgo no anticipado: el nombre de campo incorrecto en el nodo Edit Fields (criterio C7) se propagó también al nodo de mensajería (criterio C8), porque su expresión referencia ese mismo campo. La nota obtenida (91) fue más baja que la esperada si solo se penalizara C7 de forma aislada (~95), lo cual demuestra que la rúbrica evalúa correctamente la **coherencia entre nodos del workflow** y no cada nodo de manera independiente — un comportamiento deseable, ya que en un flujo real ese error haría que la automatización fallara en producción. El caso `s2_sin_json_cd2` replica, en el contexto de n8n, el mismo patrón de doble penalización observado en TP1: la falta del archivo `.json` exportado descuenta puntos dentro de C1 y además dispara el techo de `CD2`, que termina siendo el factor decisivo.

Nota de cobertura de testeo

El testeo del modo TEST en este informe tuvo **cobertura parcial**: el TP2 fue el más exhaustivo (8 entregas sintéticas, una por criterio + cumplimiento parcial). El TP1, TP5 y TP6 se testearon con 3 entregas cada uno, cubriendo 1–2 criterios y la condición de desaprobación principal. Los TP3, TP4 y TPI se sometieron al flujo de CREAR + VALIDAR + AUDITAR pero no se incluyeron sus resultados de TEST en este informe, por limitaciones de espacio. Los archivos de esas corridas (entregas sintéticas, correcciones y prompts) están disponibles en las carpetas respectivas de [Pruebas Empíricas/](#) .

9. Ajuste iterativo de consigna, rúbrica y entrega

El proceso descrito en las secciones 4 a 8 no es lineal sino **iterativo**. Si el modo TEST revela que un criterio no discrimina como se esperaba, la causa casi nunca está en el modelo que corrige, sino en el diseño de la rúbrica: una descripción ambigua, una evidencia no verificable, un peso mal distribuido o instrucciones de puntuación poco específicas.

El ciclo de trabajo aplicado en este proyecto fue, en la práctica:

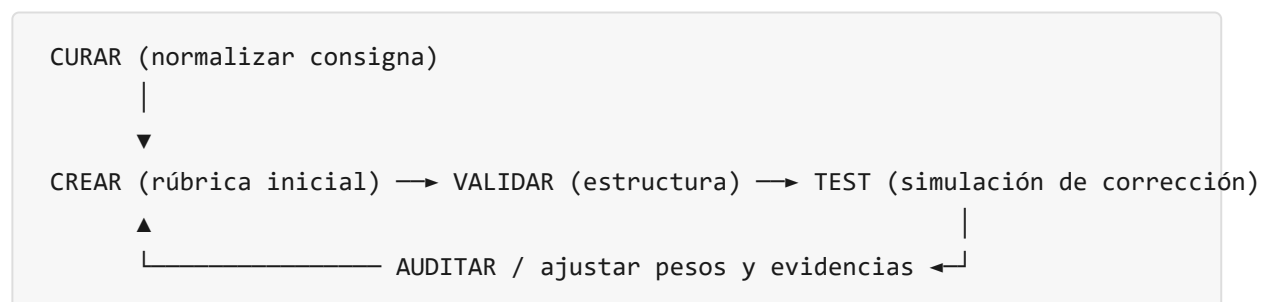


Figura 1. Ciclo iterativo de trabajo aplicado a cada rúbrica: la salida del TEST realimenta el ajuste vía AUDITAR hasta que las notas simuladas son coherentes con lo esperado.

Solo cuando el conjunto de entregas sintéticas produce notas coherentes con lo esperado para cada criterio, la rúbrica se considera lista para cargarse en el sistema real (botón "Cargar criterios" de Active-IA), junto con los campos que no forman parte del JSON y se completan aparte en el formulario (`tipo` , `numero` , `año` , `materia_id`).

Ejemplo de iteración real: rúbrica del TP3 (v1 → v2)

La primera versión de la rúbrica del TP3 (Metodologías ágiles) definía C4 ("Retrospectiva") con la siguiente `instrucciones_puntuacion` :

"Si la retrospectiva no está completa, descontar proporcionalmente."

El modo TEST con la entrega `s4_falla_c4.txt` (retrospectiva con solo dos secciones de las cuatro requeridas) produjo una nota de **88/100**, cuando se esperaba un descuento más perceptible. El diagnóstico del agente fue:

"La instrucción 'descontar proporcionalmente' no define la proporción ni el umbral mínimo. El corrector interpreto 2/4 secciones como 50% de

penalización en los 12 pts del criterio: $100 - 6 = 94$, luego otros factores lo llevaron a 88. La instrucción es ambigua."

La versión 2 reemplazó esa instrucción por:

"Por cada sección ausente de las 4 requeridas (Qué salió bien / Qué mejorar / Obstáculos / Compromisos para el próximo sprint), descontar 25% del peso del criterio (3 pts). Si ninguna sección está presente, 0 en C4."

Tras el ajuste, la misma entrega `s4_falla_c4.txt` produjo **76/100** (descuento de ~24 pts por las dos secciones ausentes), discriminando con claridad el trabajo incompleto del trabajo perfecto.

10. Caso de estudio aplicado: Organización Empresarial

A lo largo de la cursada de Organización Empresarial se aplicó este flujo de trabajo completo a las rúbricas de los distintos Trabajos Prácticos y del Trabajo Práctico Integrador (TPI), organizados dentro de la carpeta `Pruebas Empiricas/` del repositorio:

- **Pruebas Empiricas/TP1** — Git y control de versiones (`rubrica-tp1-git.json`)
- **Pruebas Empiricas/TP2** — Ingeniería de prompts y pensamiento computacional (`rubrica-tp2-ingenieria-prompts.json`)
- **Pruebas Empiricas/TP3** — Metodologías ágiles (`rubrica-tp3-metodologias-agiles.json`)
- **Pruebas Empiricas/TP4** — Ecosistema de una startup (`rubrica-tp4-ecosistema-startup.json`)
- **Pruebas Empiricas/TP5** — Live coding (`rubrica-tp5-live-coding.json`)
- **Pruebas Empiricas/TP6** — N8N (`rubrica-tp6-n8n.json`)
- **Pruebas Empiricas/TPI** — Trabajo Práctico Integrador (`rubrica-tpi-oe.json`)

Para cada una de estas rúbricas se generó, además, un conjunto de entregas de prueba (sintéticas), cada una con su correspondiente resultado de corrección (`*_correccion.json`) y el prompt real enviado al modelo (`*_prompt.txt`), lo que permite trazar de forma completa y auditable el proceso de diseño → testeo → ajuste de cada instrumento de evaluación utilizado en la materia. En los TP1 a TP6 estos archivos residen directamente en la carpeta de cada TP; en el TPI, dentro de la subcarpeta fechada `sinteticos/2026-07-01_130503/` .

Esta decisión metodológica —trabajar siempre con entregas en texto plano y evitar links, imágenes o ejecución de código como requisito de nota— es consistente con el criterio de normalización descrito en la sección 4, y es la razón por la cual estas rúbricas resultaron, en la práctica, consistentes en las pruebas de corrección automática realizadas.

Síntesis transversal del caso de estudio

Analizando el conjunto de las 7 rúbricas, emergen los siguientes patrones comunes:

- **Criterios totales:** entre 6 y 9 criterios por rúbrica (promedio ~7), más 1–2 condiciones de desaprobación (CD) en los TPs que lo requerían.
- **Patrón de ajuste más frecuente:** instrucciones de puntuación con descuentos porcentuales no definidos cuantitativamente (problema detectado en TP3 y TP4 en la primera iteración). La corrección consistió en especificar el descuento en puntos absolutos o porcentaje fijo del peso del criterio.
- **Efecto de las condiciones de desaprobación:** presente en TP1 (ausencia de `.git`), TP5 (formato de entrega incorrecto) y TP6 (ausencia del `.json` exportado). En los tres casos el techo fue el factor determinante en las pruebas de testeo, validando que estaban bien configuradas.
- **Cobertura de testeo por TP:** TP2 (8 sintéticos, todos los criterios); TP1, TP5, TP6 (3 sintéticos, criterio clave + CD); TP3, TP4 y TPI (testeo realizado, resultados disponibles en las carpetas respectivas de `Pruebas Empiricas/` pero no reportados en este informe por limitaciones de extensión).

11. Conclusiones finales

La implementación de una IA agéntica asistida por una skill especializada demuestra que es posible sistematizar la creación de instrumentos de evaluación —una tarea tradicionalmente artesanal y dependiente del criterio individual del docente— sin perder rigurosidad pedagógica, e incluso ganando en consistencia: cada rúbrica queda validada estructuralmente, curada respecto de los límites técnicos reales del corrector, y testada empíricamente contra entregas de prueba antes de usarse sobre alumnos reales.

Revisando los objetivos específicos planteados en la Introducción:

1. **Funcionamiento de la IA agéntica y el rol de la skill** (§2): documentado con las definiciones de IA agéntica, skill y sus cuatro modos. El concepto de skill como paquete de conocimiento de dominio empaquetado queda operacionalizado a lo largo de todas las secciones prácticas.
2. **Proceso completo de creación de una rúbrica** (§4–§9): documentado paso a paso, desde la normalización de la consigna hasta el ajuste iterativo post-TEST, con ejemplos aplicados y trazabilidad a archivos concretos.
3. **Límites técnicos del corrector y criterios de normalización** (§4): los cuatro límites duros del corrector (no navega externos, filtra por extensión, flujo único código/informe, no ejecuta) fueron identificados, documentados y operacionalizados en la tabla de tres destinos del modo CURAR.
4. **Validación empírica de la discriminación de criterios** (§8): verificada mediante entregas sintéticas en TP1, TP2, TP5 y TP6; todos los criterios testeados discriminaron en la dirección correcta, con cobertura parcial declarada para TP3, TP4 y TPI.
5. **Caso de estudio documentado y reproducible** (§10): las 7 rúbricas de Organización Empresarial se encuentran en el repositorio del proyecto, con sus entregas sintéticas y archivos de corrección trazables a archivos concretos.

El hallazgo conceptual más relevante de todo el proceso es que **la calidad de una corrección automática no depende principalmente del modelo de IA que corrige, sino de la calidad de la rúbrica que se le entrega**. Una rúbrica ambigua, incompleta o con requisitos que dependen de contenido que el corrector estructuralmente no puede leer (links, imágenes, ejecución) producirá correcciones

inconsistentes sin importar qué tan avanzado sea el modelo utilizado. Este hallazgo es, precisamente, el motivo por el cual la skill utilizada dedica un modo completo (CURAR) a normalizar la consigna *antes* de siquiera pensar en redactar un criterio.

Como línea de trabajo futura, sería valioso extender este análisis a otras materias con formatos de entrega más heterogéneos, evaluando qué adaptaciones adicionales necesitaría el modo CURAR para esos casos. También resultaría valioso cuantificar la dispersión de notas al correr la misma entrega varias veces (con y sin ajuste de temperatura del modelo), para evidenciar empíricamente el nivel de consistencia que la metodología propuesta logra en producción.

12. Fuentes utilizadas y reconocimiento de autoría

⚠ Nota de autoría — de lectura obligatoria

La skill `rubrica-builder` utilizada como herramienta central de este trabajo es **propiedad intelectual de Juan Cruz Robledo**, distribuida bajo licencia **Apache License 2.0** (© 2026 Juan Cruz Robledo), disponible públicamente en el repositorio: <https://github.com/JuanCruzRobledo/rubrica-builder> .

Este Trabajo Final utiliza dicha skill **exclusivamente con fines educativos y de investigación académica**, como caso de estudio para analizar y documentar la aplicación de IA agéntica a la elaboración de rúbricas de evaluación. No se reclama autoría alguna sobre el diseño, la lógica ni el código de la skill en sí misma. Todo el mérito de su concepción, arquitectura y reglas de diseño corresponde a su autor original.

Fuentes académicas

- Andrade, H. G. (2000). Using rubrics to promote thinking and learning. *Educational Leadership*, 57(5), 13–18.
- Brookhart, S. M. (2013). *How to create and use rubrics for formative assessment and grading*. ASCD.
- Stevens, D. D., & Levi, A. J. (2013). *Introduction to rubrics: An assessment tool to save grading time, convey effective feedback, and promote student learning* (2.^a ed.). Stylus Publishing.
- Zawacki-Richter, O., Marín, V. I., Bond, M., & Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education — where are the educators? *International Journal of Educational Technology in Higher Education*, 16(1), 39. <https://doi.org/10.1186/s41239-019-0171-0>
- Kasneci, E., Seßler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., ... Kasneci, G. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 102274. <https://doi.org/10.1016/j.lindif.2023.102274>.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer*

Science, 18(6), 186345. <https://doi.org/10.1007/s11704-024-40231-1>

- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *Proceedings of ICLR 2023*. <https://arxiv.org/abs/2210.03629>

Herramientas y materiales propios

- Robledo, J. C. (2026). *rubrica-builder* (skill para Claude Code). Repositorio oficial: <https://github.com/JuanCruzRobledo/rubrica-builder> . Licencia Apache-2.0. *Nota de reproducibilidad*: la versión exacta de la skill utilizada en este trabajo queda fijada en el archivo `skills-lock.json` del repositorio, con hash de contenido `45737a7cef3e144e4947046c60838e79462cd092d46e6fc28342077637706f92` , de modo que los resultados aquí documentados sean reproducibles contra esa versión y no contra una futura revisión de la skill.
- Documentación interna de la skill: `SKILL.md` , `README.md` , `references/modelo-rubrica.md` , `references/limites-corrector.md` .
- Anthropic. (2024). *Claude Code: agentic CLI for software engineering tasks*. Documentación oficial. <https://docs.anthropic.com/claude-code>
- Video propio: "¿Qué es una rúbrica?" — disponible en: drive.google.com/drive/folders/1WewN_uwqNbvnyhkBqQ4WZ2_lge4UHPCU.
- Video propio: "Normalización de la entrega para crear una rúbrica coherente y minimizar fallos" — disponible en: <https://youtu.be/qOXTPzVbJ-s>. Desarrolla el caso de estudio documentado en la sección 4.1: la decisión de estandarizar Git como tecnología de control de versiones y de reemplazar las capturas de pantalla por la carpeta `.git` como evidencia auditable.
- Material propio generado durante la cursada de Organización Empresarial: rúbricas de las Unidades 1 a 6 y del TPI, junto con sus entregas sintéticas de testeo y resultados de corrección, utilizados como caso de estudio aplicado a lo largo de este informe.